

A Startup Guide for Quantitative Financial Research

A practical research environment for Financial Machine Learning and AI

Shane Conway

Abstract

Quantitative financial research requires more than a model and a dataset. A credible result also needs a reproducible computing environment, a record of data availability and research choices, a meaningful benchmark, and a clear separation between development and assessment. This guide presents a compact workflow for beginning that work with GitHub, Colab or local Python, Jupyter, and optional coding agents. Its purpose is not to prescribe elaborate infrastructure, but to establish a durable evidence trail: what question was asked, what could have been known at the time, what procedure produced the result, and whether another researcher can reconstruct it.

Table of contents

1	The short version	2
2	Choose a working route	3
3	Know what each tool does	3
4	Start with a repository	3
5	Add a local Python environment when you need one	5
6	Give the project a shape	6
7	Add skills when the project asks for them	6
8	Treat data as part of the argument	7
9	Put a clock on every modeling row	7
10	Begin every experiment with a small baseline	8
11	Make notebooks rerunnable	8
12	Install Codex or Claude Code	9
12.1	Codex	9

12.2 Claude Code	9
12.3 Start inside the project	10
13 Give coding agents durable project instructions	10
14 Use AI as a fast collaborator	13
15 Debug one layer at a time	13
16 Keep secrets, licensed data, and large artifacts separate	14
17 Buy compute only when the experiment earns it	14
18 Build the final report while you work	15
19 A practical first session	15
Current official references	16

Quantitative research can look complicated because the finished product may contain market data, statistical models, machine learning, backtests, and a carefully designed report. The working setup is much simpler. You need a place to write code, a way to run it, a record of what changed, and enough information to reconstruct the result later.

That last point is the purpose of the whole toolchain. A useful research setup should let you answer:

- What question did I ask?
- Which data did I use, and when would those data have been available?
- Which code, package versions, and assumptions produced the result?
- Which observations helped me develop the idea, and which observations tested it?
- Can I run the analysis again from a clean start?

The goal is not to install every fashionable tool. It is to build a small, durable system in which a result can be understood, challenged, and improved.

1 The short version

If you want the smallest setup that works, use:

1. **GitHub** to store and share a versioned research project.
2. **Google Colab** to run Jupyter notebooks without configuring a computer.
3. **Python**, with NumPy, pandas, matplotlib, statsmodels, and scikit-learn.
4. **Markdown cells** to explain the question, data, assumptions, and findings.
5. **A clean-run test**: restart the notebook and run every cell from top to bottom before treating its output as evidence.

For this course, begin at the public [uc_finmlai](#) repository. Open the Week 0 setup notebook and use Colab if local Python gives you trouble. When you are ready to save your own work, connect the public course history to an empty private repository as described below. That path is sufficient for the course.

For a longer research project, a local environment is worth adding. It gives you more control, works offline, and makes it easier to move reusable code out of notebooks.

2 Choose a working route

There are three sensible routes. None makes the research more intelligent by itself.

Route	Best for	Main limitation
Colab + GitHub	Starting immediately; avoiding installation problems	The runtime is temporary and its software changes
Local Python + GitHub	Longer projects; full control; reusable code	Initial setup takes more care
Hybrid	Colab for portability, local Python for serious development	You must keep both environments consistent

The hybrid route is often the best destination. Begin in Colab, then add a local environment when the project becomes large enough that the extra control is useful. Do not spend the first week building infrastructure for a ten-line experiment.

3 Know what each tool does

The names are easier to remember when each tool has one job.

- **Python** is the language that performs the calculations.
- **A package** is reusable Python code written by someone else. NumPy handles arrays, pandas handles labeled tables and dates, statsmodels provides familiar statistical procedures, scikit-learn provides machine-learning workflows, and matplotlib draws figures.
- **An environment** is one project's private collection of Python and packages. It prevents a change for one project from silently breaking another.
- **Jupyter** is the notebook interface in which code, output, equations, and explanation can live together.
- **Colab** is a hosted Jupyter service. Google supplies the temporary computer.
- **An editor** such as VS Code displays the whole project and helps you work with notebooks, Python files, Markdown, and Git.
- **Git** records deliberate snapshots of the project on your computer.
- **GitHub** stores a remote copy of that Git history and makes collaboration and publication easier.
- **Quarto** is optional publishing software that can turn one source document containing prose and code into HTML, PDF, slides, or a website.
- **A coding agent** can inspect files, propose code changes, and run checks. It accelerates the work; it does not decide what the evidence means.

4 Start with a repository

A repository is the project folder together with its recorded history. For the course, the most reliable setup uses two remote repositories:

- `upstream` is the public course repository, from which you collect new material;
- `origin` is your private repository, to which you push your work.

Set that up as follows:

1. On GitHub, create a new **private**, **empty** repository. Do not initialize it with a README, license, or `.gitignore`.
2. Clone the public course repository, giving the local folder the name you want for your project:

```
git clone https://github.com/smc77/uc_finmlai.git YOUR-REPOSITORY
cd YOUR-REPOSITORY
```

3. Rename the existing course remote from `origin` to `upstream`, then add your empty private repository as the new `origin`:

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR-NAME/YOUR-REPOSITORY.git
git remote -v
git push -u origin main
```

4. Each week, pull the new course commit and then push the combined history to your private repository:

```
git pull upstream main
git push origin main
```

`origin` should now point to your private repository. `upstream` should point to the public course repository.

Why not simply use **Use this template**? A template is excellent when you want to begin an independent project. GitHub deliberately gives it a new history, however, so it is awkward when you need to merge weekly commits from the source repository. Cloning the course history and changing its remotes preserves the common history that makes later pulls ordinary. If you already created a template repository and have begun work, preserve that work before changing the repository setup.

The ordinary Git loop is short:

```
git pull --rebase
git status
git diff
git add path/to/file
git commit -m "Explain why the result changed"
git push
```

Read `status` and `diff` before committing. A commit should describe one change that you inspected and, when possible, tested. “Fix files” is a weak message; “Shift momentum feature behind forecast date” tells your future self why the result moved.

Git is not a backup for everything. Do not place passwords, API keys, licensed vendor data, or very large generated files in the repository. Deleting a secret in a later commit does not remove it from the earlier history.

5 Add a local Python environment when you need one

If you stay in Colab, you can skip this section initially.

For local work, install a supported Python 3 version and give each project its own environment. The standard Python route is:

```
python3 --version
python3 -m venv .venv
source .venv/bin/activate          # macOS or Linux
# .venv\Scripts\Activate.ps1      # Windows PowerShell
python -m pip install --upgrade pip
python -m pip install numpy pandas scipy matplotlib statsmodels scikit-learn jupyterlab finmlsim
```

The `python -m pip` form is deliberate: it asks the same Python interpreter to run the installer. This avoids the common problem in which pip installs a package for one Python while the notebook runs another.

`uv` is a useful modern alternative. It can manage Python versions, environments, dependencies, and lockfiles. It is fast, but it is an additional tool to learn. Standard `venv` and `pip` remain perfectly reasonable for a first project.

Whichever route you choose, record the dependencies. A `requirements.txt` file is a simple option; a `pyproject.toml` with a lockfile is stronger for a larger project. The important principle is that “I installed some packages last semester” is not an environment specification.

Confirm which Python the notebook is actually using:

```
import os, sys
print(sys.executable)
print(os.getcwd())
```

Then record the versions of the central packages:

```
import numpy as np, pandas as pd, sklearn, statsmodels

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("scikit-learn:", sklearn.__version__)
print("statsmodels:", statsmodels.__version__)
```

When an import fails, these paths and versions are more informative than “it doesn’t work.”

6 Give the project a shape

A small project does not need an elaborate architecture. This is enough:

```
my-research-project/  
  README.md  
  research_record.md  
  notebooks/  
    01_data_audit.ipynb  
    02_baseline.ipynb  
    03_model.ipynb  
  src/  
    features.py  
    evaluation.py  
  tests/  
  data/  
    README.md  
    raw/          # usually ignored by Git  
    processed/    # usually reproducible, not hand-edited  
  outputs/  
    figures/  
    tables/  
  pyproject.toml  # or requirements.txt  
  .gitignore
```

The `README.md` tells a new reader what the project asks, how to run it, where the data come from, and which output matters. The research record preserves the choices that are easy to forget: forecast time, target horizon, data cutoff, benchmark, validation design, costs, and what changed after a failed attempt.

Notebooks are good for exploration and explanation. Stable logic belongs in small Python functions once you need it more than once. This keeps a notebook from becoming a thousand-cell program whose correctness depends on the order in which someone happened to click it.

7 Add skills when the project asks for them

A few adjacent skills become useful quickly, but none needs to block the first experiment:

- **The terminal** makes paths, environments, Git, and remote machines easier to understand. Learn `pwd`, `ls`, `cd`, and how to ask a command for `--help`.
- **SQL** is the common language for selecting and joining data in databases. DuckDB is a convenient local way to practice SQL directly on CSV and Parquet files without running a database server.
- **Tests** protect reusable feature and evaluation functions. A small test that checks timestamps or a known numerical answer is more valuable than a large collection of tests that only confirm the code executes.
- **A formatter and linter**, such as Ruff, can remove distracting style errors and catch some programming mistakes. They do not test the financial logic.

- **R** remains important in statistics and empirical finance. Python is the course language, not the only legitimate research language.
- **Spreadsheets** are useful for inspecting a small table and communicating with colleagues. They are weak as the only record of a long transformation pipeline because manual edits are difficult to reconstruct.
- **A reference manager**, such as Zotero, helps preserve papers, citations, and notes. A model result should retain the intellectual sources that motivated it, not only the code that estimated it.

8 Treat data as part of the argument

In financial research, “the data” are rarely just a CSV. A useful data record includes:

- provider and dataset name;
- download or query date;
- observation range and frequency;
- units, currency, timezone, and calendar;
- identifiers and how they were linked;
- treatment of splits, dividends, delistings, and missing values;
- publication time, revision history, and the assumed availability lag;
- filters used to create the research universe;
- license or redistribution restrictions; and
- a checksum when the exact file cannot be committed.

Keep raw data immutable. A cleaning script should produce processed data; do not open the raw file in a spreadsheet, repair three cells by hand, and leave no record of what happened.

Several legitimate starting points are useful:

- [SEC EDGAR APIs](#) for public company filings and XBRL financial-statement data;
- [FRED and ALFRED](#) for macroeconomic series and historical vintages;
- the [Kenneth French Data Library](#) for factor and portfolio returns; and
- institutional services such as WRDS, CRSP, Compustat, and OptionMetrics when your university or employer has an appropriate license.

Convenient data are not automatically research-grade data. Ask whether the series includes failed securities, whether historical values have been revised, whether prices are adjusted, and whether the license permits the way you plan to store or publish the result. A ticker plus a date range is not a complete data definition.

9 Put a clock on every modeling row

The simplest rule in historical financial research is to imagine that you are alive at the decision date and the future has not happened yet.

Before modeling, write down:

1. **Decision time:** When is the forecast made?
2. **Information cutoff:** What could realistically be known then?

3. **Execution time:** When could an action actually occur?
4. **Target interval:** Which later outcome is being predicted?
5. **Evaluation period:** Which observations were not used to choose the procedure?

This catches more errors than memorizing a universal rule such as “always shift by one.” A feature computed after today’s close might be valid for tomorrow’s open and impossible for today’s close. The clock, not the function name, decides.

When practical, turn the timing story into assertions. Verify that every feature was available by the decision timestamp and that the target begins after the information cutoff. A particularly strong test is future invariance: change or delete tomorrow’s data, rebuild the pipeline, and confirm that yesterday’s feature values do not move.

10 Begin every experiment with a small baseline

Do not begin with the largest model your computer can fit. First create the smallest complete experiment:

1. define the financial question and target;
2. construct one admissible feature;
3. choose a naive or economically meaningful benchmark;
4. split forward in time;
5. fit one simple model;
6. evaluate it under the intended loss or decision metric; and
7. reconcile the forecast into a position or action, including costs when relevant.

This vertical slice proves that the timestamps, shapes, units, and evaluation code fit together. Complexity can then be added one controlled change at a time. If the simple experiment cannot run cleanly, a neural network will not repair its research design.

11 Make notebooks rerunnable

A notebook has hidden memory. A variable may remain alive after the cell that created it has been deleted. A chart may show an old model after the fitting code changes. The visible page can therefore tell a different story from the running process.

Before trusting a notebook:

1. save it;
2. restart the kernel;
3. run all cells from top to bottom;
4. inspect errors, warnings, figures, and tables; and
5. save the clean output.

Put imports, configuration, seeds, data cutoffs, and paths near the top. Avoid manual file uploads that another user cannot reproduce. Use relative paths inside the project rather than paths tied to one person’s home directory.

A random seed reconstructs one random outcome. It does not prove that the conclusion is stable. When randomness matters, repeat the experiment across seeds or independent samples

and report the variation.

12 Install Codex or Claude Code

A terminal coding agent can read a project, explain unfamiliar code, edit files, and run commands. It is optional for this course. Installing both Codex and Claude Code is also unnecessary; one is enough to learn the workflow. They use different accounts and billing arrangements, even though they work in similar ways.

Install these tools from their maintained official instructions. The commands below are current at the time of writing, but installation commands do change.

12.1 Codex

On macOS, Linux, or WSL, the standalone installer is the shortest route:

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

On Windows PowerShell, use:

```
powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

Homebrew and npm are alternatives:

```
brew install --cask codex  
# or  
npm install -g @openai/codex
```

Confirm the installation, then start Codex:

```
codex --version  
codex
```

The first launch normally opens a browser so that you can sign in. You can also start that process explicitly with `codex login` and inspect it with `codex login status`. Codex supports a ChatGPT sign-in or an OpenAI API key; API-key usage has separate API billing. Never put an API key in a notebook, prompt, Markdown file, or Git history.

12.2 Claude Code

Anthropic recommends its native installer. On macOS, Linux, or WSL, run:

```
curl -fsSL https://claude.ai/install.sh | bash
```

On Windows PowerShell, run:

```
irm https://claude.ai/install.ps1 | iex
```

Homebrew is an alternative on macOS:

```
brew install --cask claude-code
```

Confirm the installation, check its health, and start it:

```
claude --version  
claude doctor  
claude
```

The first session asks you to authenticate in a browser. Claude Code requires an eligible Claude or Anthropic Console account, or a configured supported third-party provider. The native installation updates itself; Homebrew must be updated with `brew upgrade claude-code`.

12.3 Start inside the project

Both agents use the current folder as important context. Enter the repository before launching one:

```
cd my-research-project  
git status  
codex      # or: claude
```

A good first request is read-only:

Read `README.md`, `AGENTS.md`, and `research_record.md`. Do not edit anything. Explain how to reproduce the baseline result, and list any setup or timing assumptions that remain unclear.

This tests whether the repository explains itself before an agent begins changing it.

13 Give coding agents durable project instructions

An interactive prompt tells an agent what to do **now**. An `AGENTS.md` or `CLAUDE.md` file tells it how work in this repository should generally be done. These files are useful for facts and rules that should survive across sessions:

- the research purpose and the decision being modeled;
- the data clock and other rules that must never be violated;
- the project layout and real setup or test commands;
- what must not be committed or silently replaced; and
- what evidence is required before reporting that work is complete.

They do not replace the `README`, which is written primarily for people, or the research record, which records what happened during the project. They are also not a security boundary. Keep secrets out of them and still inspect every diff and command.

Codex looks for `AGENTS.md` from the repository root toward the current working directory; a nearer file can refine the rules for one subdirectory. Claude Code uses `CLAUDE.md` files in a similar

hierarchy. Claude does not automatically read `AGENTS.md`, but a `CLAUDE.md` file can import it. If both files will always travel together, the simplest shared arrangement is therefore:

```
my-research-project/
  AGENTS.md      # the common instructions
  CLAUDE.md      # imports AGENTS.md
  README.md
  research_record.md
  ...
```

Use this tiny `CLAUDE.md`:

```
# Claude Code instructions

@AGENTS.md
```

Now the substantive instructions have one source of truth. If the files may be downloaded separately, it is also reasonable to put the full instructions in both files, provided that you keep the copies synchronized. The Week 0 folder uses that standalone approach in its copyable `AGENTS.md` and `CLAUDE.md`. Start a new agent session after changing the instructions.

Here is the complete example `AGENTS.md`:

```
# Quantitative research project instructions

## Purpose

This repository studies whether a stated information set can improve a stated financial decision. Keep the research question, not model complexity, at the center of the work. Read `README.md` and `research_record.md` before editing.

## Document clearly

- When designing something new, document it clearly in language that an advanced high school student could understand. Use a Markdown document.
- Store research results in Jupyter notebooks.

## Research invariants

- Reconstruct each historical row as if you were alive at its decision time.
- State the information cutoff, decision time, feasible execution time, target interval, units, universe, and timezone.
- Do not use later observations, revised values, or today's survivor list to construct an earlier row.
- Fit scalars, imputers, encoders, feature selection, and models on training data only. Apply the frozen fitted state to later data.
- Validate forward in time. Keep the final assessment period out of model and feature selection.
```

- Compare results with a named forecast, investment, or risk benchmark under the same target, horizon, loss, and cost assumptions.
- Keep raw data immutable. Do not silently replace missing data with synthetic data; report what is missing and stop if it changes the claim.

Working conventions

- Make the smallest change that answers the request. Preserve unrelated work.
- Keep notebooks readable as research narratives. Move reused logic into tested functions under ``src/``.
- Use project-relative paths, explicit random seeds, and named configuration values rather than hidden notebook state.
- Record material changes to data, timing, features, targets, benchmarks, or evaluation in ``research_record.md``.
- Do not add dependencies, download large data, or change the research question without explaining the need first.
- Never commit credentials, ``.env`` values, licensed raw data, or private employer material.

Verification

- Find the repository's real setup and test commands in ``README.md``, ``pyproject.toml``, or the task runner. Do not invent successful checks.
- Run the smallest relevant test while working and the documented full check before reporting completion.
- Restart and run any affected notebook from top to bottom in a clean kernel.
- For time-dependent features, test future invariance: changing data after a cutoff must not alter feature values before that cutoff.
- Inspect ``git diff`` and report changed files, tests run, assumptions, and remaining limitations. If a check could not run, say why.

Review priorities

When reviewing code, look first for timing leakage, preprocessing fitted before the split, invalid random shuffling, revised or survivor-biased data, overlapping labels treated as independent, undisclosed search, weak benchmarks, and omitted trading costs.

Simplicity first

- Use the minimum code that solves the problem. Do not reinvent the wheel when a widely used tool already does the job well.

Goal-driven execution

- Define success criteria and work until they are verified. Turn large tasks

into verifiable goals with intermediate checks. If something is unclear, ask for more detail.

Good project instructions are short, concrete, and true. “Write good code” is too vague. “Restart and run the affected notebook from top to bottom” can be checked. Delete instructions that become stale, and use a nested instruction file only when one part of a large repository genuinely needs different rules.

14 Use AI as a fast collaborator

Chat tools and coding agents are exceptionally useful for:

- explaining an unfamiliar error;
- locating relevant code in a repository;
- drafting a small function or test;
- translating an idea into a first implementation;
- reviewing a diff for edge cases;
- improving documentation; and
- proposing alternative visualizations or robustness checks.

They are most useful when the task is bounded. A good request states:

- the objective;
- the files in scope;
- what must not change;
- the data and timing assumptions;
- what successful output looks like; and
- which command or test will check the result.

Then inspect the diff. Run the code. Read the output. An agent can write a perfectly polished backtest with an impossible timestamp, silently replace a missing dataset with synthetic data, or report success after testing the wrong file. Fluency is not evidence.

Never paste private data, credentials, unreleased research, or employer code into a service unless its terms and your organization’s policy allow it. Record material AI assistance in the project disclosure. The human researcher remains responsible for the question, permissions, evidence, and final claim.

15 Debug one layer at a time

When something fails, preserve the exact error before trying several repairs. Then identify the layer:

1. **Location:** Am I in the expected folder? Print `pwd`, `os.getcwd()`, and list the directory.
2. **Interpreter:** Which Python is running? Print `sys.executable`.
3. **Environment:** Are the expected packages installed in that interpreter?
4. **Data:** Does the file exist? Are columns, index, units, and dates what the code expects?
5. **Method:** Are array shapes, missing values, and model assumptions valid?

6. **Research design:** Even if the code runs, could the decision maker have known the inputs?

Read the final line of a Python traceback first, then move upward to the first line of your own code. Reduce the problem to one import, one file read, or one small calculation. A minimal failing example is easier for you, another person, or an AI assistant to diagnose.

16 Keep secrets, licensed data, and large artifacts separate

A useful `.gitignore` often begins with:

```
.venv/  
.env  
.ipynb_checkpoints/  
__pycache__/  
.DS_Store  
data/raw/  
outputs/cache/
```

Store API keys in environment variables or a secrets manager, not in code. A local `.env` file may be convenient, but it must be ignored by Git and replaced by a harmless `.env.example` that lists variable names without values.

Large datasets and videos do not belong in ordinary Git history. Store them in an approved data service or object store and record how to obtain them. Git LFS can help with permitted large files, but it does not change a vendor's license.

Back up the repository and the irreplaceable data record. Generated figures and caches should be rebuildable; raw licensed data may need separate protected storage.

17 Buy compute only when the experiment earns it

Most tabular financial research needs a good CPU, adequate memory, and fast storage—not a GPU. A GPU becomes useful for larger neural networks, language models, and some simulation workloads. Colab can provide temporary accelerators, but availability and limits vary.

Before paying for more compute:

- profile the code rather than guessing;
- test the pipeline on a smaller time range or universe;
- cache expensive raw-data transformations;
- vectorize clear array operations;
- avoid refitting the same object unnecessarily; and
- create a documented fast mode for ordinary development.

The full run should be reserved for a frozen configuration. Faster hardware can multiply experiments, which also multiplies the opportunity to select a lucky result.

Docker can capture a full software environment and is valuable for deployment or complex system dependencies. It is optional for a first research project. An isolated Python environment plus a lockfile is usually enough.

18 Build the final report while you work

Do not wait until the end to reconstruct the story. Maintain:

- a one-paragraph question and decision context;
- a data and timestamp ledger;
- a baseline table;
- a development-versus-assessment distinction;
- a log of important searches and failed ideas;
- a small number of figures that each answer a question; and
- a limitations section that says what the evidence does not establish.

A strong portfolio repository should let a reader find the main result quickly, then trace it back to code and assumptions. Include a README, a clean notebook or report, setup instructions, a data-access explanation, and an AI-use disclosure. Publish only data, papers, figures, and code that you are permitted to redistribute.

Quarto is a good optional final layer because it can render code and explanation from one source. A conventional notebook exported to HTML is also acceptable. Presentation polish matters, but a polished number with no reconstructable procedure is decoration rather than research.

19 A practical first session

You can establish the whole baseline in about an hour:

1. Create an empty private repository on GitHub.
2. Clone the course repository and connect your private repository as `origin`.
3. Open `lectures/week00/setup.ipynb` in Colab.
4. Run it from top to bottom and preserve the environment summary.
5. Confirm that `origin` is yours and `upstream` is the course repository.
6. Make one harmless Markdown change, inspect it with `git diff`, commit it, and push it.
7. Create a `work/` folder and one short research-record entry.
8. If using a coding agent, copy and customize the example `AGENTS.md` and `CLAUDE.md`, then begin with the read-only repository request above.
9. Ask the agent to explain one cell, inspect the answer, and record how you checked it.

You are ready when you can produce four pieces of evidence:

- the URL of your repository and a successful commit hash;
- one notebook that passes restart-and-run-all;
- the Python executable and key package versions used for that run; and
- a short statement of the data clock: what was known, when the decision was made, and what later outcome was measured.

That is enough infrastructure. The next step is not another installation. It is a financial question worth answering.

Current official references

Setup instructions change, so prefer these maintained sources over commands in an old recording:

- [GitHub: connecting to a repository](#)
- [GitHub: repositories created from templates](#)
- [Python: virtual environments and packages](#)
- [Jupyter installation](#)
- [Google Colab FAQ](#)
- [VS Code Python guide](#)
- [uv documentation](#)
- [Quarto: get started](#)
- [OpenAI Codex CLI quickstart](#)
- [Codex authentication](#)
- [Codex AGENTS.md instructions](#)
- [Claude Code installation](#)
- [Claude Code project instructions](#)
- [SEC EDGAR data APIs](#)
- [FRED API documentation](#)
- [Kenneth French Data Library](#)